MULTIMODAL INTERACTIVE TECHNOLOGIES AND APPLICATIONS

Botanical Garden

Report for Heuristic Evaluation

Group Name	Gardeners
Kaifei Xu	11115439
Keyan Cheng	11154362
Shiqi Huang	11111452
Yufei Liu	11118413

May 25, 2026

1 Project Concept

1.1 Overview

Botanical Garden is a browser-based botanical garden that combines **immersive panoramic exploration** with a **conversational, voice-driven assistant**. Users move through 360° garden scenes, look at or select 3D plant models, and ask natural-language questions. The goal is to turn a static botanical exhibit into a fluid multimodal experience where visual context, plant selection, speech input, adaptive information panels and TTS feedback work together.

1.2 Characterizing Features

- **Immersive 3D environment.** The prototype renders linked 360° panoramic garden scenes with navigation portals and 3D plant models. This gives users a spatial context for exploration.
- **Multimodal input.** The system combines visual context, plant selection, typed input, speech-to-text via the Web Speech API, and click/touch fallback controls. Users can therefore interact by looking, selecting, speaking, typing, or clicking overlay options depending on the situation.
- **Multimodal output.** Responses are presented through a coordinated combination of 3D plant highlights, adaptive 2D information panels (botanical, medicinal, comparison and fallback), and text-to-speech narration.
- **LLM-assisted dialogue with local fallback.** A lightweight stateless backend provides JSON endpoints for intent parsing, information generation, disambiguation prompts and comparison speech. The prototype first attempts LLM-assisted generation when enabled, but keeps local deterministic parsing and structured plant data as fallback so that the core interaction does not break when the LLM is unavailable.
- **Referential ambiguity handling.** When the user asks an ambiguous question such as *"what is this"* while multiple plants are visible or otherwise plausible in the current scene, the system enters a referential ambiguity flow. Candidate plants are labelled A / B / C and can be resolved by clicking, typing, or speaking a letter or plant name, with retry guidance and direct-click fallback.
- **Contextual comparison.** The system remembers recently visited plants and can compare the current selection with a named plant, a previously visited plant, or a plant found in the global database. In the current prototype, comparison focuses on drought tolerance while the panel also shows supporting attributes where available.

1.3 Target Users and Why It Is Multimodal

The system targets **botanical garden visitors, botany students and users who are unable to visit in person** exploring a botanical collection on desktop. Instead of reading long static labels, users can ask focused questions about the plant in front of them.

Botanical Garden is multimodal **by design**: users interact through multiple input modalities, including visual context, click/raycast selection, typed text and speech input, while the system responds through multiple output modalities, including 3D spatial highlights, visual information cards and spoken TTS feedback.

2 Main Tasks

This document defines the three main interaction tasks used for heuristic evaluation of the Botanical Garden prototype.

2.1 Task 1 - Visual Detection and Multimodal Disambiguation

Goal. The user wants to identify a plant using an ambiguous deictic question such as *"what is this"* or *"what plant am I looking at?"*. The task evaluates whether the system can detect that the reference is unclear and guide the user to the intended plant.

Modalities. *Input:* current camera view, speech-to-text, typed text, and click/touch selection on the A / B / C overlay. *Output:* candidate highlights in the 3D scene, labelled overlay options, information panel, and TTS feedback.

Flow.

1. The user asks an ambiguous identification question, for example *"what is this"* or *"what plant am I looking at?"*.
2. If no plant is already selected, the frontend checks the plants in the current visual field. When multiple visible plants are detected, the system enters a referential ambiguity flow. If visibility detection cannot determine a single clear target and the current scene contains multiple candidate plants, the scene candidates are used as a fallback ambiguity set.
3. The system generates A / B / C labels for the candidate plants, highlights them in the 3D scene, and displays a clarification overlay. A backend LLM call may be used to produce a natural clarification prompt; if this fails, a local prompt is used.
4. The user resolves the ambiguity by clicking one candidate, typing a letter, speaking a letter, or saying the plant name.
5. If the answer cannot be matched, the system asks the user to choose again. After repeated failures, it guides the user toward directly clicking the intended target.
6. Once a candidate is resolved, the selected plant is highlighted and its information card is shown.

Success criteria. Ambiguity is triggered when the user uses an unclear reference and more than one plant is visible or otherwise plausible in the current scene; every candidate is clearly labelled; clicking, typing, and voice-based letter selection work; failure cases provide a retry path and a direct-click fallback; no plant-specific information card appears before the target is resolved.

2.2 Task 2 - Plant Selection and In-depth Dialogue

Goal. The user selects one plant and asks targeted follow-up questions about its properties, such as basic botanical information or medicinal value. The task evaluates whether the system keeps the selected plant in context and presents the right type of information panel.

Modalities. *Input:* plant selection by click/raycast, speech or typed natural-language query, and fallback buttons for supported topics. *Output:* selected-plant highlight, botanical or medicinal information panel, fallback panel for unsupported interests, and TTS narration.

Flow.

1. The user selects a plant directly, or resolves Task 1 ambiguity so that one plant becomes the current selection.

2. The user asks a property question, for example *"what is its medicinal value?"*, *"tell me about this plant"*, or *"what is it used for?"*.

3. The frontend parses the intent and topic of interest. For medicinal value, medical, or herbal-use questions, the system opens the dedicated medicinal information panel.

4. For supported botanical queries, the system displays the general plant information card. Information generation first attempts to use the backend LLM endpoint; if generation fails or is disabled, the UI still falls back to structured local plant data.

5. If the user asks for an unsupported interest, such as a topic not available in the current prototype, the system shows a fallback interaction that guides the user toward supported topics such as botanical features or medicinal value.

6. Subsequent follow-up turns remain attached to the selected plant until the user selects another plant or closes the plant panel.

Success criteria. The selected plant remains visually highlighted; medicinal questions open the medicinal panel; general botanical questions open the standard information card; unsupported interests do not dead-end but offer supported alternatives; local fallback prevents the core interaction from breaking when LLM generation is unavailable.

2.3 Task 3 - Contextual Plant Comparison

Goal. The user compares the currently selected plant with another plant mentioned by name or inferred from visit history. For example:

```
Is this more drought tolerant than the ficus?
```

Flow.

1. The user has a current plant selected and asks a comparison question, such as *"Is this more drought tolerant than the ficus?"* or *"compare it with the previous plant"*.

2. The frontend parses the comparison intent and resolves the second plant. It first checks explicit plant names and aliases, then uses the user's visited plant history for references such as *"the previous one"*. If the target was not visited but exists in the plant database, it can still be used as a global database fallback.

3. The comparison view displays both plant descriptions and key attributes, including drought tolerance, height, lifespan, and medicinal value where available.

4. The frontend sends both resolved plant records to the backend `/api/compare` endpoint when LLM comparison speech is enabled. The backend is responsible for generating a natural-language comparison answer, while the frontend remains responsible for rendering the structured side-by-side result.

5. If no valid comparison target can be resolved, the system explains the problem and asks the user to select another plant or use a known plant name.

Success criteria. The system resolves comparison targets from explicit names, aliases, recent history, or the global plant database; the comparison panel shows both plants in a structured layout; drought tolerance is compared clearly in the current prototype; missing targets produce a useful error instead of a silent failure; the spoken summary and visual comparison remain consistent.

3 Prototype Architecture

3.1 Architectural Overview

Botanical Garden is implemented as a browser-based prototype with a lightweight local backend. The frontend provides the immersive garden experience, plant selection, speech interaction, visual feedback, and information panels. The backend provides optional LLM-assisted intent parsing and response generation, while the frontend keeps a deterministic local fallback so that the main interaction loop can still work when the LLM service is unavailable.

At a high level, the prototype is divided into four layers:

1. **Immersive scene layer:** renders the panoramic garden, 3D plant models, and spatial highlights.
2. **Interaction layer:** captures pointer-based plant selection, typed input, speech-to-text input, visible-plant ambiguity detection, and ambiguity-resolution actions.
3. **Dialogue and grounding layer:** maps user utterances to intents, connects those intents to the currently selected plant or recently visited plants, and decides which UI response should be shown.
4. **Knowledge and generation layer:** stores structured plant records and, when enabled, uses the backend LLM service to generate more natural spoken explanations.

The architecture intentionally keeps the most important interaction state on the client side. This makes the prototype responsive and allows plant selection, local intent parsing, panel rendering, and fallback behavior to work without a persistent server session.

3.2 Frontend Architecture

The frontend is a Vite application written in modular JavaScript. The entry point is `src/main.js`, which initializes global UI controls, speech recognition, plant selection, scene navigation, and the first garden scene.

The main frontend modules are:

- `src/scene/*`: renders the 360-degree garden scenes, 3D plant models, and plant selection in the immersive scene.
- `src/app/*`: owns interaction state and high-level behavior such as identifying a plant, querying an attribute, comparing plants, and resolving ambiguity.
- `src/intent/*`: parses user utterances locally and optionally delegates to the backend intent API.

- `src/ui/*`: renders the plant information panel, medical panel, comparison panel, fallback panel, ambiguity overlay, and query controls.
- `src/speech/speech.js`: integrates browser speech recognition and text-to-speech output.
- `src/data/*`: contains local scene, plant, and intent data used by the prototype.

The frontend state is centralized in `src/app/state.js`. It tracks the active panoramic scene, the currently selected plant, the set of plants currently visible in the user's view, ambiguity candidates derived from those visible plants, the ordered list of visited plant ids, and the speech recognition failure count for retry behavior.

This state is small but important. It allows the system to interpret phrases such as "this plant", "the previous one", or "compare it with lavender" against the user's actual interaction history instead of treating each utterance as an isolated command.

3.3 Backend Architecture

The backend is a minimal Node.js HTTP server located in `server/index.js`. It exposes JSON endpoints that support the frontend dialogue flow:

- `GET /api/health`: health check and configured model information.
- `POST /api/parse-intent`: parses a user utterance into an intent and slots.
- `POST /api/generate-info`: generates a spoken answer for one plant.
- `POST /api/generate-disambiguation`: generates a clarification prompt for multiple candidate plants.
- `POST /api/clarify`: resolves a user's answer to an ambiguity prompt.
- `POST /api/compare`: generates a comparison response for two plants.

The backend is intentionally stateless from the frontend's perspective. Each request includes the context needed for that turn, such as the utterance, selected plant, visited plant ids, candidate plants, or dialogue history. This keeps the prototype simple to run locally and avoids hidden server-side state that would be hard to debug during evaluation.

The backend services are split by responsibility:

- `server/services/intentParser.js`: deterministic parser fallback.
- `server/services/llm.js`: LLM-backed parsing and response generation.
- `server/services/plantDb.js`: reads the plant database from `src/data/plants.json`.
- `server/handlers/*`: request handlers for query, clarify, and compare flows.

If the LLM call fails, the server returns a local fallback or an empty generated speech field. The frontend can still render structured cards from local plant data, so the visual part of the interaction remains available.

3.4 Core Interaction Flow

The most common interaction starts when the user looks at a plant area and asks a question.

1. The system checks the plants that are currently visible in the user's view.
2. If only one plant is visible or clearly selected, the frontend records it as the current plant; if several visible plants could match the user's reference, the frontend opens an ambiguity flow.
3. The user submits text through typing or browser speech recognition.
4. `handleQuery()` sends the utterance and current context through the intent parser.
5. The parsed intent is routed to one of the frontend handlers: identification, attribute query, comparison, ambiguity resolution, or unknown intent.
6. The selected plant is visually highlighted in the 3D scene.
7. The corresponding 2D panel is rendered: botanical information, medicinal information, comparison result, fallback options, or ambiguity choices.
8. Text-to-speech reads the concise answer or clarification prompt.

For ambiguous references, the frontend stores the visible candidate plants in state and shows an A/B/C overlay. The user may resolve the ambiguity by clicking an option or by speaking a phrase such as "A" or the plant name. As a fallback option, users can directly click a plant model to make an explicit selection.

For comparison, the frontend combines the current selected plant with the visited plant history. This enables contextual queries such as "Is this more drought tolerant than the previous one?" The comparison handler resolves the second plant, chooses the relevant attribute, and renders a side-by-side comparison panel.

3.5 Data Model and Assets

Plant knowledge is stored as structured JSON in `src/data/plants.json`. Each plant record contains identifiers, display names, aliases, descriptions, attributes, and optional information such as medicinal value, toxicity, or drought tolerance. Scene metadata is stored in `src/data/scenes.json`, while intent-related rules are stored in `src/data/intents.json`.

Visual assets are kept in the `public/` directory:

- panoramic scene images;
- 3D plant models in `.glb` format;
- UI assets.

This separation keeps the prototype extensible. New plants can be added by combining a plant data record, a model asset, and scene placement metadata, without changing the main interaction code.

4 Code Repository

The source code of the prototype is available in the project GitHub repository:

```
https://github.com/Shiqi-H/MITA
```

To run the prototype locally, the required dependencies should first be installed from the project root:

```
npm install
```

The frontend and backend must then be started in two separate terminal windows. The frontend can be launched with:

```
npm.cmd run dev
```

The backend server can be launched from another terminal with:

```
npm.cmd run server
```

After both processes are running, the prototype can be accessed from a web browser through the local development URL printed by Vite in the frontend terminal, usually:

```
http://localhost:5173/
```

If a different port is shown in the terminal, the user should open that printed local URL instead. Vite's development server listens on `localhost` by default, and `localhost` is allowed as a development host by default.

5 Demo Video

A short demo video is available at the following link:

```
https://www.youtube.com/watch?v=luDSMh8OVXc
```